

FMCG Inventory Data Integration Pipeline

Project Report

Python · Pandas · SQLite — a 10-day self-study build

Prepared for: AmirReza BabaAhmadi

Data as of: 1 September 2026

Report date: 5 September 2026

Contents

1. Executive Summary
2. Business Problem & Objectives
3. Data Sources
4. Pipeline Architecture
5. Data Quality Findings (Q1)
6. Business Rule Validation (Q2)
7. Standardization & Transformation (Q3)
8. Data Integration (Q4)
9. Business-Ready Analytical View (Q5)
10. Operational Flags (Q6)
11. Loading & Reproducibility (Q7)
12. Logging & Audit Trail (Q8)
13. Limitations
14. Conclusion & Recommendations

1. Executive Summary

This project builds a complete, reproducible ETL pipeline for a regional FMCG (fast-moving consumer goods) distributor's sales, order, and inventory data. Four raw CSV extracts — sales, orders, inventory snapshots, and a warehouse master file — were profiled, validated against eight explicit business rules, cleaned and standardized, safely integrated into one business-ready analytic table, screened for four operational risk conditions, and loaded into a SQLite database. Every step is logged for audit.

The pipeline processed 1,877 raw sales rows, 767 raw order rows, and 3,470 raw inventory-snapshot rows across 8 warehouses. After validation and cleaning, 1,427 sales rows, 633 order rows, and 2,933 inventory rows were loaded, producing a business-ready view of 360 SKU/warehouse combinations. Every SQLite table's row count was verified against its source DataFrame, and zero fact-table rows reference a warehouse that doesn't exist in the warehouse master.

Key findings: 36.9% of SKU/warehouse combinations show excess inventory relative to recent sales velocity, 13.1% show no sales movement at all in the trailing 30 days (dead stock), and 6.4% have open order quantities that exceed what's currently on hand. These are flagged for business review, not acted on automatically — see Section 8.

2. Business Problem & Objectives

A distributor tracks sales, purchase orders, and warehouse inventory in three separate systems that export flat files on different schedules with no shared data-entry standard. Before anyone can answer basic operating questions — which SKUs are close to stocking out, which warehouses are overstocked, whether open orders are large enough to cover recent demand — the four extracts have to be profiled, cleaned to a common standard, safely joined, and checked for internal consistency.

This project answers eight concrete questions with evidence rather than assumptions:

- Q1 — Can the raw data be trusted, and where exactly is it dirty?
- Q2 — Which records violate the business's own data rules?
- Q3 — Can the four files be standardized automatically, without manual spreadsheet edits?
- Q4 — Can the sources be merged without silently losing or duplicating records?
- Q5 — What does a single, business-ready view of inventory look like?
- Q6 — Which SKU/warehouse combinations need operational attention right now?
- Q7 — Is the load into SQLite reproducible and internally consistent?
- Q8 — Is there a full audit trail of what happened and when?

3. Data Sources

No production export was available for this project, so a synthetic-but-realistic extract was generated (script: `scripts_generate_raw_data.py`, seeded with numpy for full reproducibility). The generator follows a combined approach: plausible transaction/order/inventory table structures paired with a simulated 8-warehouse distribution network, deliberately seeded with the same categories of problems a real-world export would carry, so the pipeline has something genuine to clean rather than a tidy toy dataset.

Problems injected on purpose:

- Inconsistent column headers across files ("SKU", "Warehouse_ID", "OrderedQty", "SnapshotDate", ...)
- Mixed date formats (ISO, US, "15-Jan-2026") and outright blank dates
- Blank SKUs and blank quantities
- Unflagged negative quantities — the source systems carry no "this is a documented return" indicator
- Warehouse IDs with inconsistent casing/formatting ("wh-01", "WH_05")
- Warehouse IDs that don't exist anywhere (WH-99) or that appear in transactions but not in the warehouse master (WH-09) — a dangling foreign key
- Exact duplicate rows and duplicate primary keys (re-sent transactions/orders)
- Order-status text with typos and inconsistent casing ("Shiped", "PENDING", "cancelled")

4. Pipeline Architecture

The pipeline is organized into five modules under `src/`, run in a fixed order by `src/main.py`:

- `extract.py` — reads the four raw CSVs exactly as they arrive
- `profile.py` — full data-quality profile of every raw file (Q1)
- `transform.py` — standardization, validation rules, safe merges, the business-ready view, and the operational flags (Q2–Q6)
- `load.py` — writes five tables to SQLite and verifies the load (Q7)
- `main.py` — orchestrates all of the above and writes logs, figures, and the quality summary (Q8)

Every file goes through the exact same fixed sequence: read raw → standardize column names → convert types (dates, numbers) → enforce the warehouse foreign key → apply the remaining validation rules → drop exact duplicates → de-duplicate on the primary key. Re-using one set of transform functions across all four files (rather than writing bespoke cleaning code per file) is what makes the pipeline maintainable and its behavior predictable.

5. Data Quality Findings (Q1)

Every raw file was profiled before any cleaning took place: row/column counts, exact column names as received, missing-value percentages, exact duplicate rows, duplicate primary keys, unparseable dates, negative values in quantity columns, blank SKUs, and warehouse IDs absent from the warehouse master. Full detail is logged to `logs/pipeline.log`; the headline numbers:

File	Rows	Exact duplicates	Blank/ missing SKU	Unparseable dates	Negative quantities
sales.csv	1877	27	24	329	35
orders.csv	767	7	0	95	20
inventory.csv	3470	25	0	355	118
warehouses.csv	8	0	—	—	—

Table 1. Raw data-quality profile, before any cleaning.

Warehouse-ID orphans (values that don't resolve to any row in warehouses.csv) were also counted per file: 27 in sales, 14 in orders, and 69 in inventory. These are addressed under Rule R2 in the next section rather than dropped ad hoc.

Column headers as received also confirm the standardization problem directly: sales.csv arrived with headers "SKU", "SaleDate " and "Warehouse_ID" side by side — three different spacing/casing conventions in one six-column file. This is exactly what Section 7 (Q3) standardizes automatically.

6. Business Rule Validation (Q2)

Eight rules were defined before any cleaning code was written, so that every drop/flag decision downstream traces back to an explicit, documented rule rather than an ad hoc judgment call made while looking at the data:

Rule	Statement	Enforcement
R1	SKU must not be blank	Rows dropped; count logged
R2	warehouse_id must exist in the warehouse master table	Rows quarantined to data/processed/quarantine_*.csv — not silently dropped
R3	Date columns must be parseable	Unparseable values coerced to null via <code>pd.to_datetime(errors='coerce')</code> ; rows with a required null date dropped
R4	Quantity-type columns must be numeric	Non-numeric values coerced to null via <code>pd.to_numeric(errors='coerce')</code>
R5	Negative quantity is only valid if flagged as a documented return	No return flag exists in the source data, so any negative quantity/ordered_qty is treated as an error and dropped
R6	on_hand_qty cannot be negative	Rows floored out; count logged
R7	Primary keys should not repeat	transaction_id / order_id de-duplicated, keep-first
R8	Required columns must not be entirely null	Verified during profiling (Section 5)

Table 2. Business rules validated by `src/transform.py`.

R2 deserves a specific callout: rather than silently dropping rows with an unrecognized warehouse ID, the pipeline routes them to a quarantine file per source (`data/processed/quarantine_sales_bad_warehouse_id.csv`, etc.) with every original column intact, so a data steward can review and, where appropriate, correct and re-ingest them. Nothing disappears without a trace.

7. Standardization & Transformation (Q3)

Six reusable functions live once in `src/transform.py` and are applied identically to every file, so a fix or improvement only needs to be made in one place:

- `standardize_column_names` — lower-case, strip whitespace, collapse internal spaces to underscores, strip stray punctuation

- `normalize_ids` — upper-case, trim, unify separators to a dash (so "wh-01", " WH-01 " and "WH_05" all resolve to a single canonical form)
- `parse_dates` — `pd.to_datetime(..., errors='coerce')`, applied consistently regardless of the source format mix
- `to_numeric_safe` — `pd.to_numeric(..., errors='coerce')` for every quantity-like column
- `clean_text` — trims and lower-cases free-text fields (order status), with a small typo map ("shiped" → "shipped") applied before validating against the canonical status set
- `drop_exact_duplicates` — removes bit-for-bit duplicate rows, with the count logged per file

Applying the same six functions everywhere, in the same order, is what turns three inconsistently-formatted CSVs into tables that can be safely joined — the alternative (bespoke per-file cleaning scripts) doesn't scale and is hard to audit.

8. Data Integration (Q4)

Grain was fixed before any merge was written: one row per (sku, warehouse_id) for the business-ready view, one row per transaction/order for the fact tables. Every merge uses `how='left'` with `indicator=True`; row counts before and after each merge are logged, and unmatched rows are captured separately rather than dropped silently.

Merge	Rows before	Rows after	Unmatched	Interpretation
Analytic ← Inventory	360	360	0	Every SKU/warehouse combination has a current stock snapshot
Analytic ← Sales	360	360	47	No sales activity in the trailing 30 days — benign, not an error
Analytic ← Orders	360	360	214	No open orders in the trailing 30 days — benign, not an error
Analytic ← Warehouses	360	360	0	Zero orphans, because R2 quarantined bad warehouse IDs before this stage

Table 3. Merge accounting for the business-ready analytic view.

The row count staying at 360 before and after every merge confirms there was no fan-out (no accidental one-to-many join inflating the row count) — a check the pipeline makes automatically and would raise a warning for if it failed.

9. Business-Ready Analytical View (Q5)

The `analytic_sku_warehouse` table has one row per (sku, warehouse_id) — 360 rows in the current run — with the following columns:

- `on_hand_qty` — most recent inventory snapshot for that SKU/warehouse
- `recent_sales_qty`, `last_sale_date` — trailing 30-day sales volume and most recent sale

- recent_orders_qty, last_order_date — trailing 30-day open-order volume (cancelled orders excluded) and most recent order
- warehouse_name, city, region — joined from the warehouse master
- four operational flag columns — see Section 10

This is the table a category manager or operations analyst would actually query day to day, rather than joining the three raw fact tables by hand each time.

10. Operational Flags (Q6)

Four simple, non-optimized flags were computed directly on the business-ready view, deliberately favoring transparency over sophistication for this first pass:

Flag	Definition	Rows	% of 360
Stockout risk (F1)	On-hand quantity ≤ 0 while there was recent sales demand	1	0.3%
Excess inventory (F2)	On-hand quantity is more than 4× the recent sales quantity	133	36.9%
Order backlog (F3)	Open order quantity exceeds the quantity currently on hand	23	6.4%
Dead stock (F4)	Stock is on hand but there were zero sales in the last 30 days	47	13.1%

Table 4. Operational flags computed in `src/transform.add_operational_flags()`.

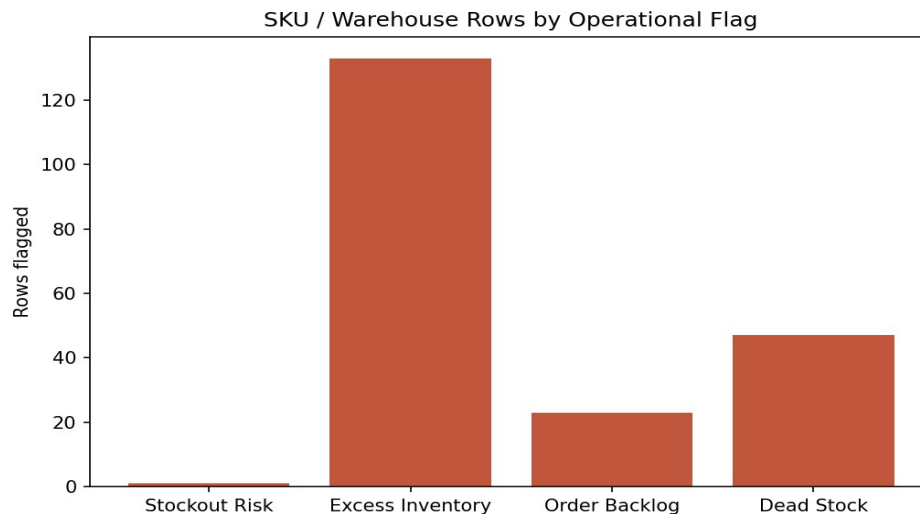


Figure 1. SKU/warehouse rows by operational flag.

Excess inventory is the largest single flag at 36.9% of SKU/warehouse combinations — a reasonable first-pass signal given a 4× threshold, but one that should be reviewed against real service-level targets before it drives action (see Limitations, Section 13). Only one combination shows outright stockout risk, and 13.1% show no sales movement at all in the trailing 30 days.

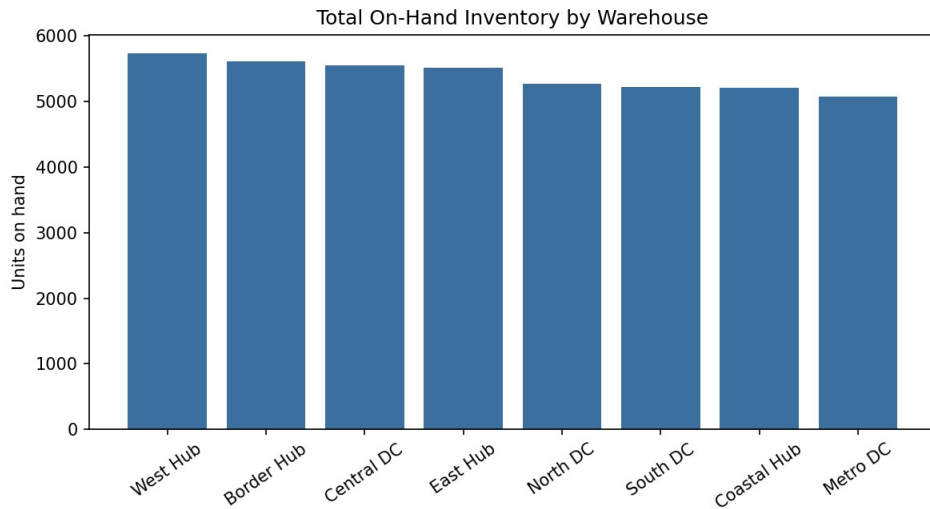


Figure 2. Total on-hand inventory by warehouse.

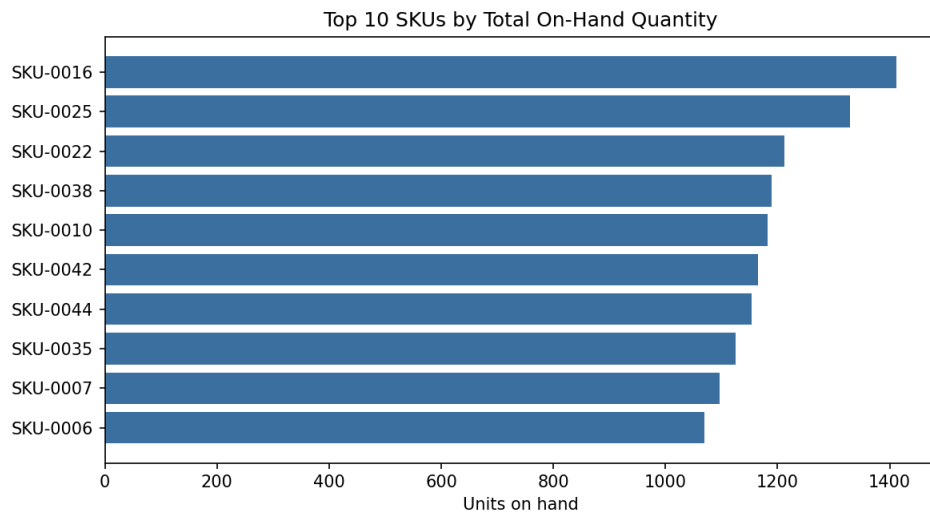


Figure 3. Top 10 SKUs by total on-hand quantity.

11. Loading & Reproducibility (Q7)

Five tables were written to database/inventory.db with if_exists='replace': dim_warehouses, fact_sales, fact_orders, fact_inventory, and analytic_sku_warehouse. After loading, the pipeline runs its own quality checks rather than assuming the write succeeded:

Table	Rows in DataFrame	Rows in SQLite	Match
dim_warehouses	8	8	Yes
fact_sales	1427	1427	Yes
fact_orders	633	633	Yes
fact_inventory	2933	2933	Yes

Table	Rows in DataFrame	Rows in SQLite	Match
analytic_sku_warehouse	360	360	Yes

Table 5. Post-load row-count verification (DataFrame vs. `SELECT COUNT(*)`).

A second check confirms referential integrity end to end: 0 rows in `fact_sales` reference a `warehouse_id` absent from `dim_warehouses` (out of 4,993 total rows across the three fact tables).

The pipeline uses only relative (pathlib) paths, runs end to end with a single command (`python -m src.main`), and is idempotent — running it twice in a row from a clean data/processed/ and database/ produces identical row counts and identical flag counts, since the source generator is seeded and every transform is deterministic.

12. Logging & Audit Trail (Q8)

`logs/pipeline.log` is overwritten (not appended) on every run and records, with a timestamp on every line: pipeline start and end; rows read per raw file; every profiling statistic from Section 5; every validation rule violation and quarantine action from Section 6; row counts dropped by cleaning and by exact-duplicate removal; merge before/after/unmatched counts from Section 8; every operational flag count from Section 10; and the post-load verification results from Section 11. Any unhandled exception would be captured with its full traceback rather than failing silently.

This makes the log file itself a complete, chronological answer to "what did the pipeline actually do on this run" — useful both for debugging and for demonstrating to a reviewer that no step was skipped.

13. Limitations

- The underlying data is synthetic, not a live production export; absolute magnitudes (units, revenue) are illustrative, not real sales figures.
- No source system carries a "this negative quantity is a documented return" indicator, so Rule R5 treats every negative quantity as a data-entry error. A real source system would carry a return/refund flag that should be honored instead of a blanket drop.
- The excess-inventory (4×) and order-backlog (>on-hand) thresholds in Section 10 are reasonable starting points for a first pass, not thresholds tuned against real service-level agreements — they should be reviewed with the business before being used to trigger purchasing or allocation decisions.
- The trailing window is fixed at 30 days of recent activity over 70 days of history; both are configurable constants in `src/main.py`, not hard business requirements.
- Quarantined rows (R2) are written to disk for review but are not automatically corrected or re-ingested — that step is intentionally a human decision.

14. Conclusion & Recommendations

The pipeline meets all eight objectives set out in Section 2: raw data quality is fully profiled and documented, business rules are explicit and enforced with an audit trail, the four sources are standardized and merged without silent data loss, a single business-ready view answers real operating questions, four operational risk conditions are flagged transparently, the SQLite load is verified rather than assumed, and every run is fully logged and reproducible.

Recommended next steps: validate the excess-inventory and order-backlog thresholds against the business's actual service-level targets; add a documented return/refund indicator to the sales extract so genuine returns are no longer indistinguishable from data-entry errors; and review the quarantined rows in data/processed/ with whoever owns the warehouse master data, since a recurring dangling warehouse ID usually points to a synchronization gap between source systems rather than a one-off typo.